

State Aggregation →

- ∅ Treats certain states as the same. (make group)
- ∅ When we update the value of any state in a group, the values of other states in that group is updated.

∅ State aggregation is another example of linear function approximation. There is one feature for each group of states. Each feature will be 1 if the current state belongs to the associated group, and 0 otherwise.

State	Value			
s_1	0	}	<u>Group 1</u>	$x(s) = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$
s_2	0			
s_3	0			
s_4	0			
s_5	0	}	<u>Group 2</u>	$x(s) = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$
s_6	0			
s_7	0			
s_8	0			

$\hat{v}(s, w) = w_1$

$\hat{v}(s, w) = w_2$

- ∅ The approximate value of a state is the weight associated with the group that state belongs to.

How to Compute Gradient of
Monte Carlo with State Aggregation

$$W \leftarrow W + \alpha (G_t - \hat{v}(s_t, W)) \nabla \hat{v}(s_t, W) \quad (43)$$

Since the state aggregation is an example of linear function approximation, so the gradient is equal to the feature vector. Therefore, the update rule modifies only the weight corresponding to the current active group.

$$\left. \begin{aligned} w_i &\leftarrow w_i + \alpha [G_t - \hat{v}(s_t, W)] \times 0 \\ w_j &\leftarrow w_j + \alpha [G_t - \hat{v}(s_t, W)] \times 1 \end{aligned} \right\} \nabla \hat{v}(s_t, W) = X(s_t) = \begin{bmatrix} 0 \\ \vdots \\ 0 \\ 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

State aggregation forces states in the same group to use the same value estimate.

Semi-Gradient TD for Policy Evaluation →

∅ TD learning is built on the idea that the agent can use its own estimate to update its prediction.

∅ Gradient Monte Carlo update our current value estimate to be closer to a sample of the return G_t .

$$W \leftarrow W + \alpha [G_t - \hat{v}(s_t, W)] \nabla \hat{v}(s_t, W)$$

∅ we can replace the return in this update with an estimate of the value.

$$w \leftarrow w + \alpha [U_t - \hat{v}(s_t, w)] \nabla \hat{v}(s_t, w)$$

✗ If U_t is an unbiased estimator of the true value then our function approximator $\hat{v}_w$ converge to a local optimum under the appropriate conditions!

✗ We can also replace $U_t \doteq R_{t+1} + \gamma \hat{v}(s_{t+1}, w)$ i.e. with a bootstrap target, such as one step TD target.

✗ This is still an estimate of the return, but in this case, the estimate is biased. The TD target uses the current value estimate, which will likely not equal the true value function.

✗ Because of this we can't guarantee this algorithm will converge to a local minimum of the value error.

✗ However, the upside is that TD target often has lower variance than the sample of the return, which means TD will tend to converge in fewer updates.

U_t biased $\rightarrow$ $\hat{v}_w$ may not converge to a local optimum.

✗ The TD update is not actually a stochastic gradient descent update!

→ TD is a Semi-gradient Method

$$\nabla \frac{1}{2} [u_t - \hat{v}(s_t, w)]^2$$

To see why? let's take the gradient of the error with respect to the weights for one sample!

where $u_t = R_{t+1} + \gamma \hat{v}(s_{t+1}, w)$

$$= (u_t - \hat{v}(s_t, w)) [\nabla u_t - \nabla \hat{v}(s_t, w)]$$

$$\neq -(u_t - \hat{v}(s_t, w)) \nabla \hat{v}(s_t, w)$$

doesn't look like TD update!

If $\nabla u_t = 0$, then above both expressions will be equal. However, in TD the target contains the estimate of the value, which depends on the weight. This means $\nabla u_t \neq 0$ for TD.

$$\nabla u_t = \nabla (R_{t+1} + \gamma \hat{v}(s_{t+1}, w))$$

$$= \gamma \nabla \hat{v}(s_{t+1}, w)$$

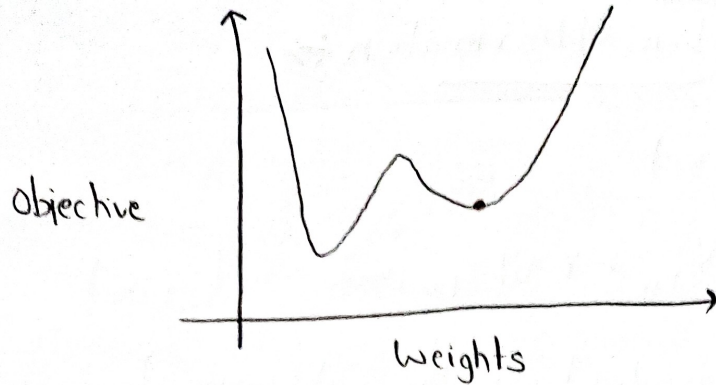
$$\neq 0$$

Therefore TD is not performing gradient descent updates on squared error. We call this as

Semi-gradient!

Semi-gradient TD(0) for estimating $\hat{v} = v_{\pi}$

- 1) Input : the policy π to be evaluated.
- 2) Input : a differentiable function $\hat{v} : \mathcal{S}^+ \times \mathbb{R}^d \rightarrow \mathbb{R}$
such that $\hat{v}(\text{terminal};) = 0$
- 3) Algorithm parameter : Step size $\alpha > 0$
- 4) Initialize value function weights $w \in \mathbb{R}^d$ arbitrarily
(e.g $w = 0$)
- 5) Loop for each episode :
 - 6) Initialize S
 - 7) Loop for each step of episode :
 - 8) Choose $A \sim \pi(\cdot | S)$
 - 9) Take action A and observe R, S' .
 - 10) $w \leftarrow w + \alpha [R + \gamma \hat{v}(S', w) - \hat{v}(S, w)] \nabla \hat{v}(S, w)$
 - 11) $S \leftarrow S'$
 - 12) Until S is terminal.



$$\emptyset \quad W \leftarrow W + \alpha \left[\underbrace{G_t - \hat{v}(s_t, W)}_{\text{target}} \right] \nabla \hat{v}(s_t, W)$$

Under reasonable assumptions, gradient Monte Carlo will approach a local minimum of mean squared value error with more and more samples.

This is because it uses an unbiased estimate of the gradient of the value error.

$$\emptyset \quad W \leftarrow W + \alpha \left[\underbrace{R_{t+1} + \gamma \hat{v}(s_{t+1}, W) - \hat{v}(s_t, W)}_{\text{target}} \right] \nabla \hat{v}(s_t, W)$$

TD target depends on our estimate of the value in next state. This means our update could be biased because the estimate in our target may not be accurate.

We cannot guarantee that semi gradient TD will converge to a local minimum of the mean squared value error.

However, TD learns faster than Monte Carlo (MC), because TD can learn during the episode and has lower variance episodes.

TD update with linear function Approximation $\Rightarrow$

$$w \leftarrow w + \alpha \delta_t \nabla \hat{v}(s_t, w)$$

$$\text{where } \delta_t \doteq R_{t+1} + \gamma \hat{v}(s_{t+1}, w) - \hat{v}(s_t, w)$$

- ✗ In the linear case, the gradient of the approximate value is just the feature vector for that state.

$$\hat{v}(s_t, w) \doteq w^T x(s_t)$$

$$\nabla \hat{v}(s_t, w) = x(s_t)$$

- ✗ We have already shown how linear value approximation is a strict generalisation of a tabular value function approximation.
- ✗ We can also show that linear TD is strict generalization of both Tabular TD and TD with state aggregation.

1) Tabular TD —

$$x(s_i) = \begin{bmatrix} 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{bmatrix}$$

i^{th} element $\rightarrow$

$$\hat{v}(s_i, w) = w_i$$

(# for tabular TD, the value approximation for a state is equal to the weight associated with the current state.)

$\Rightarrow$ We can therefore, think of the weight vector as just a table of the values, one for each state.

(46)

$$w \leftarrow w + \alpha (R_{t+1} + \gamma \hat{v}(s_{t+1}, w) - \hat{v}(s_t, w)) \cdot X(s_t)$$

Therefore for Tabular TD, the feature vector $X(s_t)$ selects a single weight associated with the current state.

$$w_i \leftarrow w_i + \alpha (R_{t+1} + \gamma \hat{v}(s_{t+1}, w) - \hat{v}(s_t, w))$$

↑ this weight is just

value ~~estimate~~ estimate for the state for tabular TD

⇒ Therefore this update corresponds to the tabular TD update we saw in previous course.

✗ Why do we Care about the special Case of linear function approximation?

⇒ True Objective for TD →

✗ TD update with Linear function approximation

$$w_{t+1} = w_t + \alpha [R_{t+1} + \gamma \hat{v}(s_{t+1}, w_t) - \hat{v}(s_t, w_t)] X(s_t)$$

$$\text{Where, } \hat{v}(s, w) = w^T X(s)$$

$$w_{t+1} = w_t + \alpha \left[R_{t+1} + \gamma w_t^T x_{t+1} - w_t^T x_t \right] x_t$$

$$w_{t+1} = w_t + \alpha \left[R_{t+1} x_t + \underbrace{\gamma w_t^T x_{t+1} x_t}_{\text{Scalar}} - \underbrace{w_t^T x_t x_t}_{\text{Scalar}} \right]$$

$$w_{t+1} = w_t + \alpha \left[R_{t+1} x_t + x_t \left\{ \gamma w_t^T x_{t+1} - w_t^T x_t \right\} \right]$$

$$w_{t+1} = w_t + \alpha \left[R_{t+1} x_t - x_t \left\{ x_t^T w_t - \gamma x_{t+1}^T w_t \right\} \right]$$

$$w_{t+1} = w_t + \alpha \left[R_{t+1} x_t - x_t (x_t - \gamma x_{t+1})^T w_t \right]$$

$\Rightarrow$ What this update look like in expectation? The expected update is key for proving convergence

The TD update can be rewritten as the expected update plus a noise term. The expected update characterises the ~~the~~ expected change in the weight from one time step to the next.

$$E[\Delta w_t] = \alpha (b - A w_t)$$

$$\text{where } b \doteq E[R_{t+1} x_t]$$

$$A \doteq E[x_t (x_t - \gamma x_{t+1})^T]$$

The weights are said to converge, when this } why?
expected TD update is 0

$$E[\Delta w_{TD}] = \alpha (b - A w_{TD}) = 0 \Rightarrow \left\{ w_{TD} = A^{-1} b \right\}$$

∅ We call this solution as
TD fixed point.

(47)

∅ It can be proven that linear TD converges to w_{TD} .
In fact w_{TD} minimizes

$$(b - Aw)^T (b - Aw)$$

∅ This objective extends the connection b/w TD and bellman equations, to the function approximation setting.

In tabular setting, we describe TD as sample based method for solving bellman equation.

Linear TD similarly approximates the solution to the bellman equation, minimizing what is called the projected bellman error.

The key takeaway is that even though TD doesn't converge to the minimum of the mean squared error, it does converge to the minimum of principled objective, based on bellman equations.

Reason why TD fixed point ' w_{TD} ' isn't equal to the minimum ~~error~~ value error solution because of the bootstrapping under function approximation. If our approximate of next state is persistently inaccurate due to function approximation, then TD forever update towards inaccurate target.